Ministry of Education of Republic of Moldova

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Software Engineering Department

Embedded Systems

Laboratory work work #2.2

Preemtive multitasking and inter-task communication. FreeRTOS.

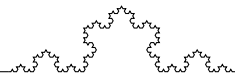
During the preparation of this report, the author used various AI Tools such as ChatGPT, Gemini, Claude to generate/augment the content, generate the code and structure the directory. The resulting information was reviewed, validated, and adjusted to meet the requirements of the laboratory assignment.

Author: Timur Cravtov
std. gr. FAF-231

Verified by:
Alexei Martîniuc
asist. univ

Chișinău, 2026

Contents



1 Objective and Technical Analysis

The objective of this laboratory work is to study real-time operating systems for embedded systems by implementing a multitasking monitor system on a microcontroller using the `FreeRTOS` real-time operating system. The system manages multiple concurrent tasks, including button state monitoring, statistics calculation, LED signaling, and periodic reporting via UART. The implementation utilizes a *preemptive* scheduling approach, where the FreeRTOS kernel can interrupt a running task to give control to a higher-priority one.

Through this work, the following learning outcomes are pursued:

- Understanding the foundations of preemptive scheduling and priority-based task management with FreeRTOS.
- Creating and managing FreeRTOS tasks using `xTaskCreate()` and `vTaskStartScheduler()`.
- Mastering inter-task synchronization using FreeRTOS semaphores (`SemaphoreHandle_t`) and mutexes.
- Utilizing FreeRTOS timing primitives such as `vTaskDelay()` and `vTaskDelayUntil()` for periodic execution.
- Developing a modular software architecture (MCAL/ECAL/APP) to ensure code portability and maintainability.

1.1 FreeRTOS and Preemptive Multitasking

FreeRTOS is a real-time operating system kernel designed for embedded microcontrollers [4]. Unlike bare-metal cooperative schedulers where tasks must explicitly yield, FreeRTOS provides *preemptive* multitasking: the kernel maintains a tick interrupt (typically 1ms) and can forcefully switch context from a lower-priority task to a higher-priority one at any tick boundary [3]. Each task has its own stack and execution context, managed entirely by the kernel.

Key differences from cooperative (bare-metal) scheduling:

- Tasks can block (e.g., waiting on a semaphore) without stalling the entire system.
- Priority-based scheduling ensures time-critical tasks are serviced first.
- Synchronization primitives (mutexes, semaphores) are required to safely share data between tasks.
- The kernel handles stack allocation and context switching transparently.

1.2 FreeRTOS Task Creation and Scheduling

Tasks in FreeRTOS are created using `xTaskCreate()`, which allocates a stack, assigns a priority, and registers the task with the scheduler [3]. Each task is implemented as an infinite loop function with the signature `void taskFunction(void* parameters)`. The scheduler is started by calling `vTaskStartScheduler()`, after which the kernel takes control of task execution.

The task priorities used in this implementation are:

- **Priority 3 (highest)**: Reading Task – must respond quickly to button state changes.
- **Priority 2**: Statistics Task – processes press data when signaled.
- **Priority 1 (lowest)**: Display Task – periodic reporting does not require urgency.

Figure 1 illustrates how the FreeRTOS kernel schedules these three tasks over time. Tasks alternate between running (consuming CPU) and blocked/sleeping states (consuming zero CPU). The kernel's tick interrupt triggers context switches when a higher-priority task becomes ready.

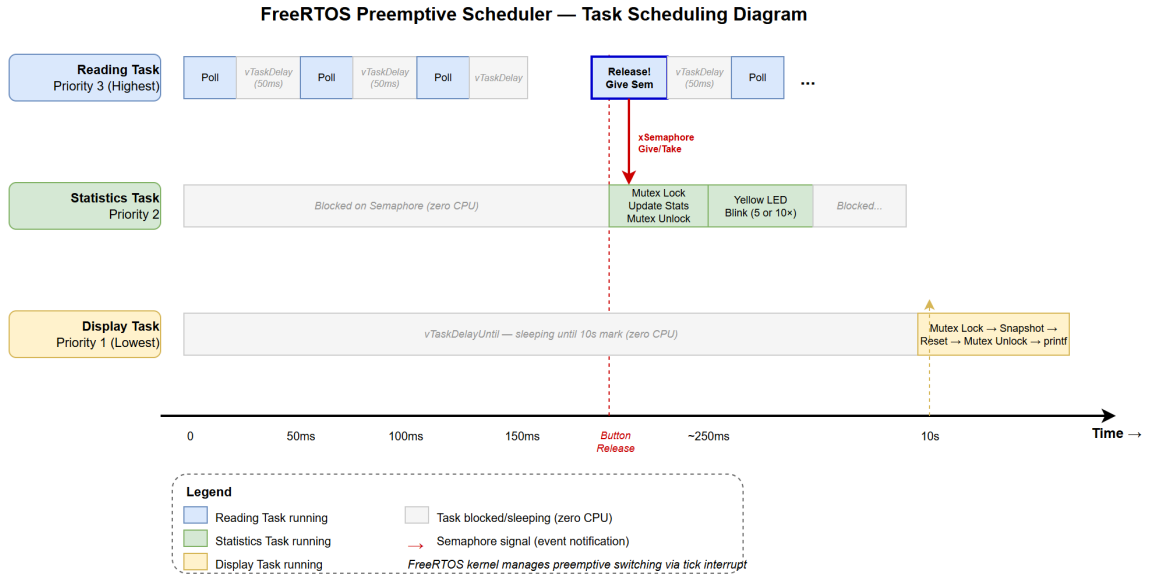
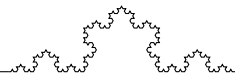


Figure 1: FreeRTOS Task Scheduling Diagram

1.3 Inter-Task Communication: Semaphores and Mutexes

FreeRTOS provides several synchronization mechanisms. In this implementation, two are used [3]:

- **Binary Semaphore** (`pressSemaphore`): Used to signal the Statistics Task that a new button press has been detected. The Reading Task calls `xSemaphoreGive()`



upon button release, and the Statistics Task blocks on `xSemaphoreTake()` with `portMAX_DELAY`, waking only when a press event occurs. This is far more efficient than polling a flag, as the blocked task consumes no CPU time.

- **Mutex** (`statsMutex`): Protects the shared statistics variables (press counts and durations) from concurrent access by the Statistics Task and Display Task. The mutex ensures that the Display Task reads a consistent snapshot of the data before resetting the counters.

1.4 FreeRTOS Timing: vTaskDelay and vTaskDelayUntil

FreeRTOS provides two primary delay functions for periodic task execution [3]:

- **vTaskDelay(ticks)**: Delays the task for a specified number of ticks *relative to the call*. Used in the Reading Task for polling at approximately 50ms intervals.
- **vTaskDelayUntil(&lastWakeTime, ticks)**: Delays until an *absolute* tick count, compensating for the task's own execution time. Used in the Display Task to achieve precise 10-second reporting intervals regardless of how long the `printf` call takes.

2 System Design

2.1 System Architecture

Figure 2 illustrates the overall architecture of the multitasking monitor system. The core components include:

- **Microcontroller (ATmega328P)**: The central processing unit running FreeRTOS, which manages task scheduling, context switching, and synchronization.
- **Button**: A digital input device used to capture user interactions. It is connected with an external pull-down resistor, allowing for simple state detection (pressed vs. released).
- **LEDs**: Three LEDs (Green, Red, Yellow) provide visual feedback. The Green and Red LEDs indicate the type of last button press (short or long), while the Yellow LED blinks rapidly to signal each new press event (5 blinks for short, 10 for long).
- **UART Interface**: Used for serial communication to output periodic statistics to the STDIO interface. Configured at 9600 baud.
- **FreeRTOS Kernel**: Manages task creation, preemptive scheduling, semaphores, and mutexes. The kernel tick provides the timebase for all delay and timing operations.

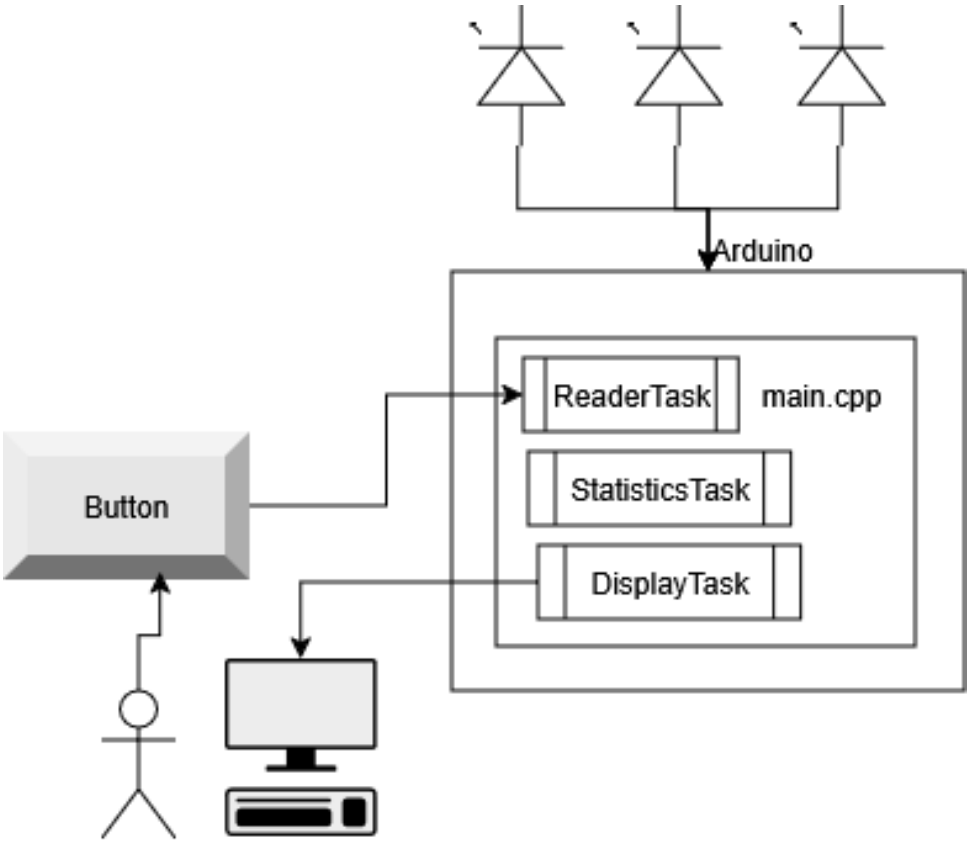


Figure 2: System Architecture Diagram

2.2 Flowchart of the program

2.2.1 Individual Task Flowcharts

Figure 3 depicts the flow of the Reading Task (priority 3), which is responsible for monitoring the button state and measuring press durations. The task polls the button every 50ms using `vTaskDelay()`, tracking state transitions via a `lastState` variable. When a rising edge is detected (button pressed), the current time is recorded. When a falling edge is detected (button released), the duration is calculated and stored in `ReadingTask::lastDuration`. The task then signals the Statistics Task via `xSemaphoreGive(pressSem` and updates the LED visualization (green for short press, red for long press).

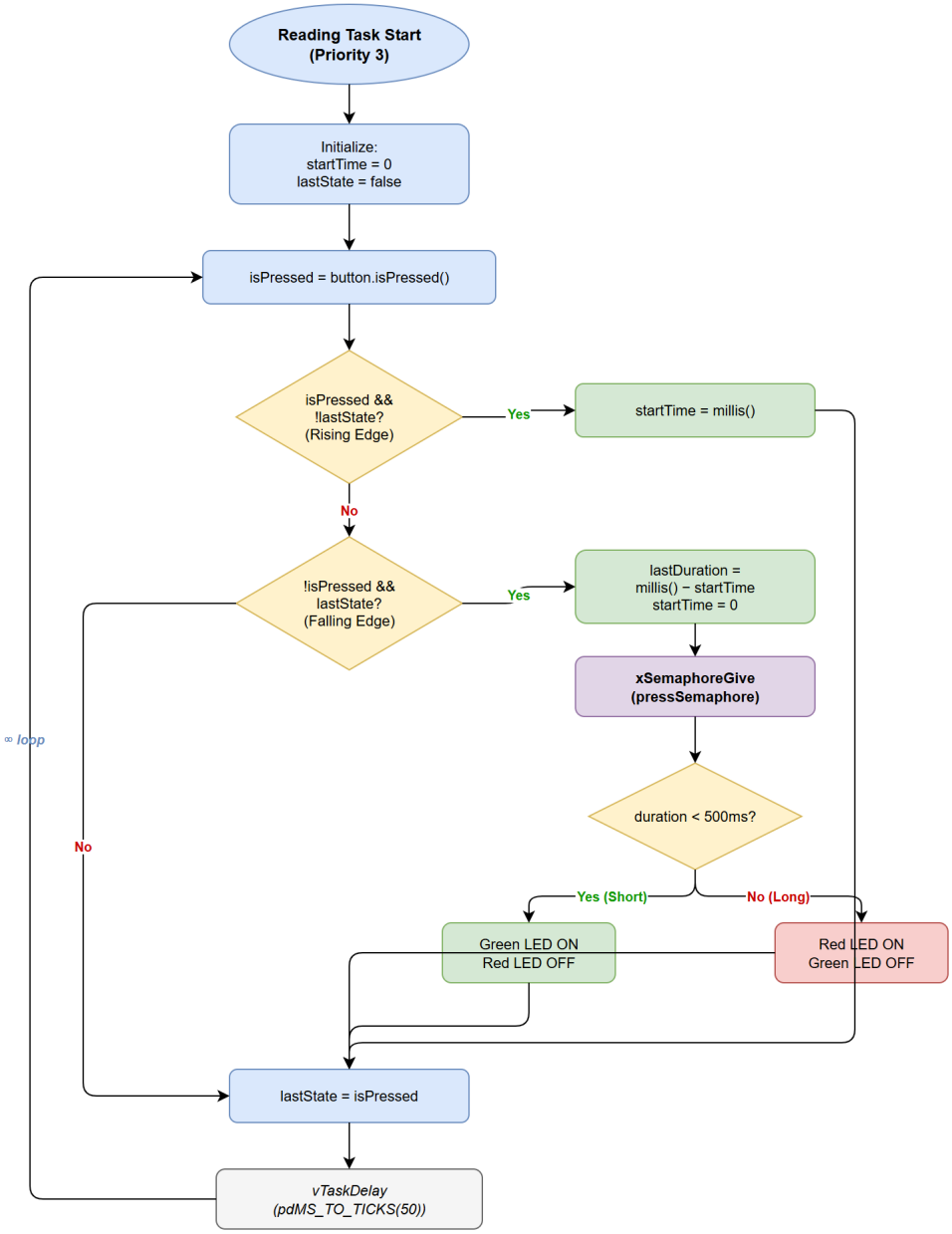


Figure 3: Reading Task Flowchart

Figure 4 illustrates the flow of the Statistics Task (priority 2). It blocks on `xSemaphoreTake(pressS portMAX_DELAY)`, consuming zero CPU while waiting. When signaled by the Reading Task, it acquires the `statsMutex`, updates the appropriate counters (short/long press count and cumulative duration), then releases the mutex. Afterwards, it performs a rapid blink sequence on the Yellow LED: 5 blinks (100ms on/off each) for a short press, or 10 blinks for a long press.

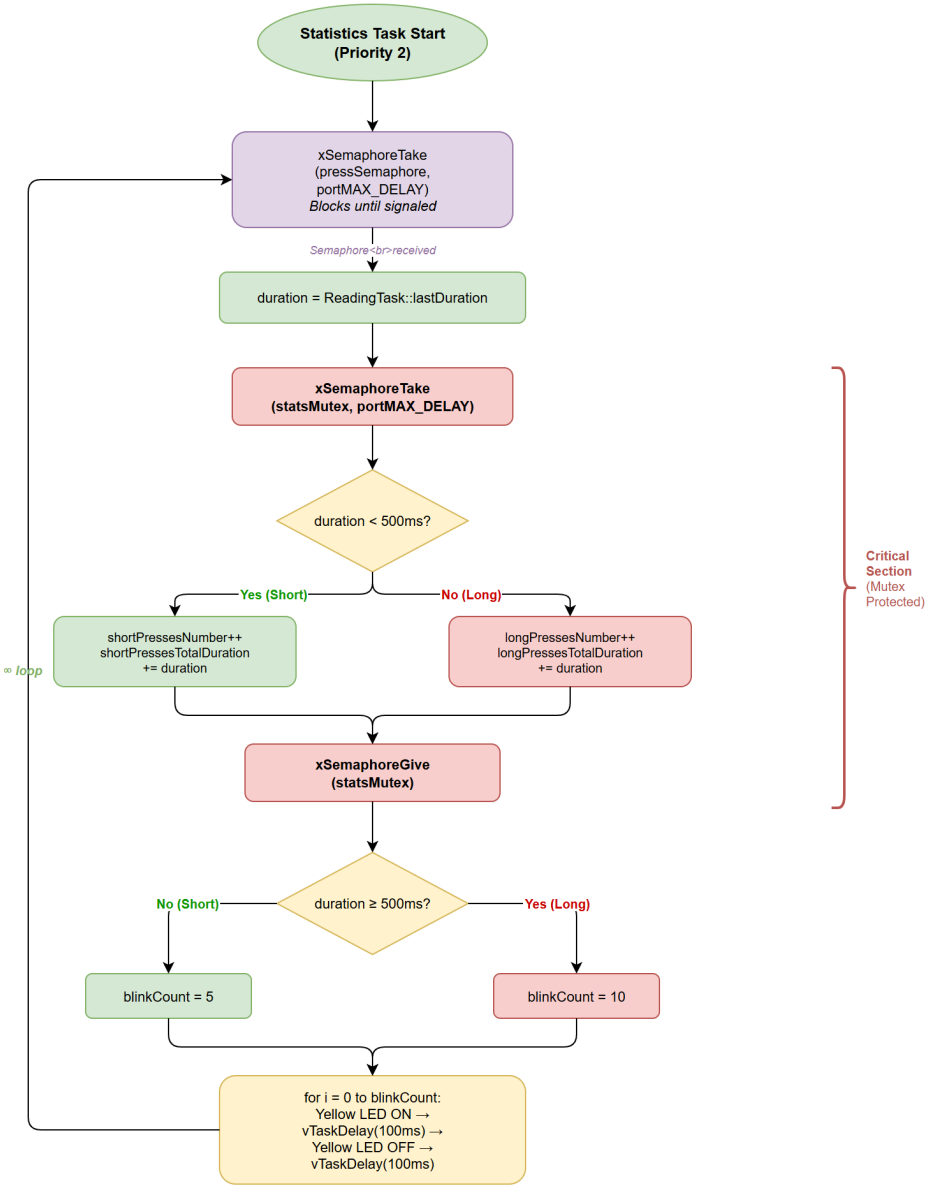


Figure 4: Statistics Task Flowchart

Figure 5 shows the flow of the Display Task (priority 1), which runs every 10 seconds using `vTaskDelayUntil()` for precise timing. It acquires the `statsMutex`, takes a snapshot of the current statistics, resets all counters to zero, then releases the mutex. It calculates the average press duration and outputs the results via UART using `printf_P()`.

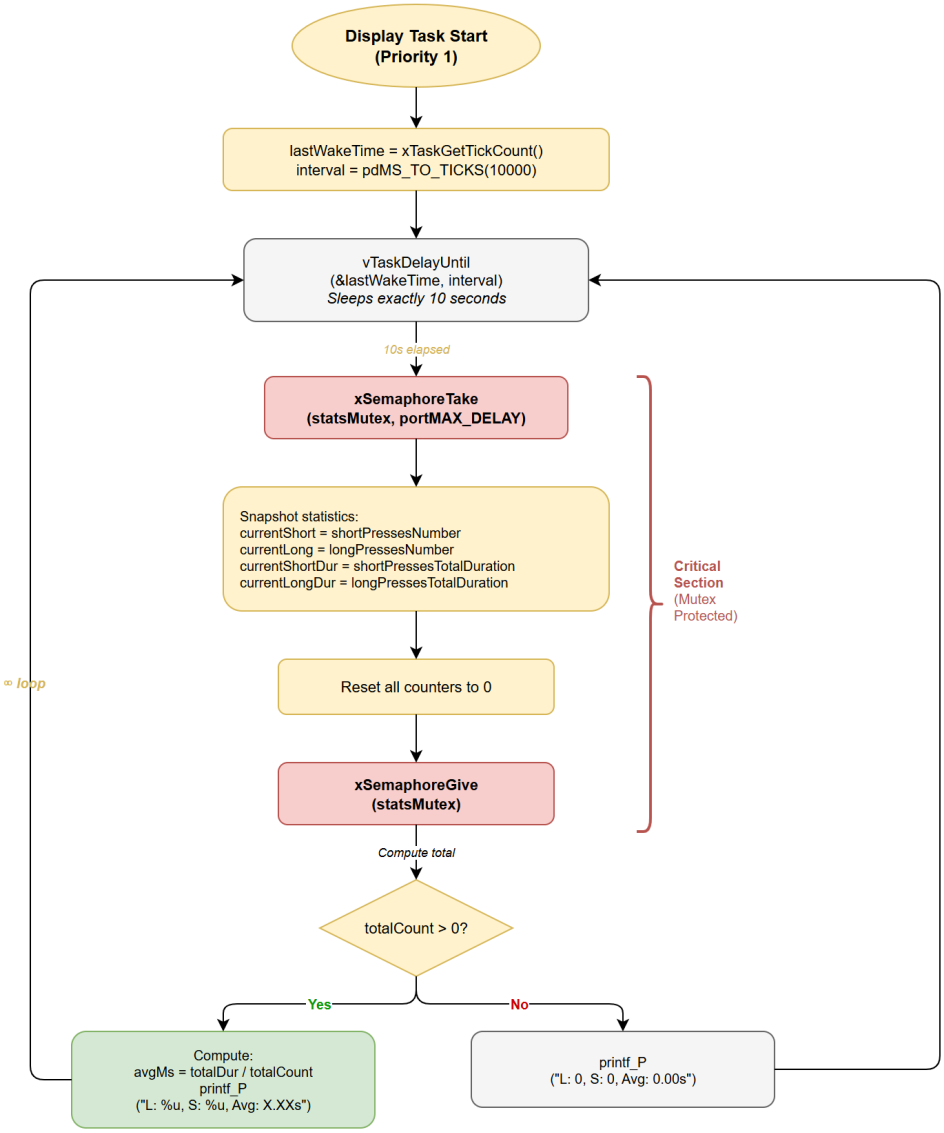


Figure 5: Display Task Flowchart

2.2.2 General System Flowchart

Figure 6 provides an overview of the entire system's flow. It shows how the three FreeRTOS tasks interact through synchronization primitives: the binary semaphore signals press events from Reading to Statistics, while the mutex protects shared data between Statistics and Display. The FreeRTOS kernel handles preemptive scheduling based on task priorities, ensuring the Reading Task always gets immediate CPU time when it needs to sample the button.

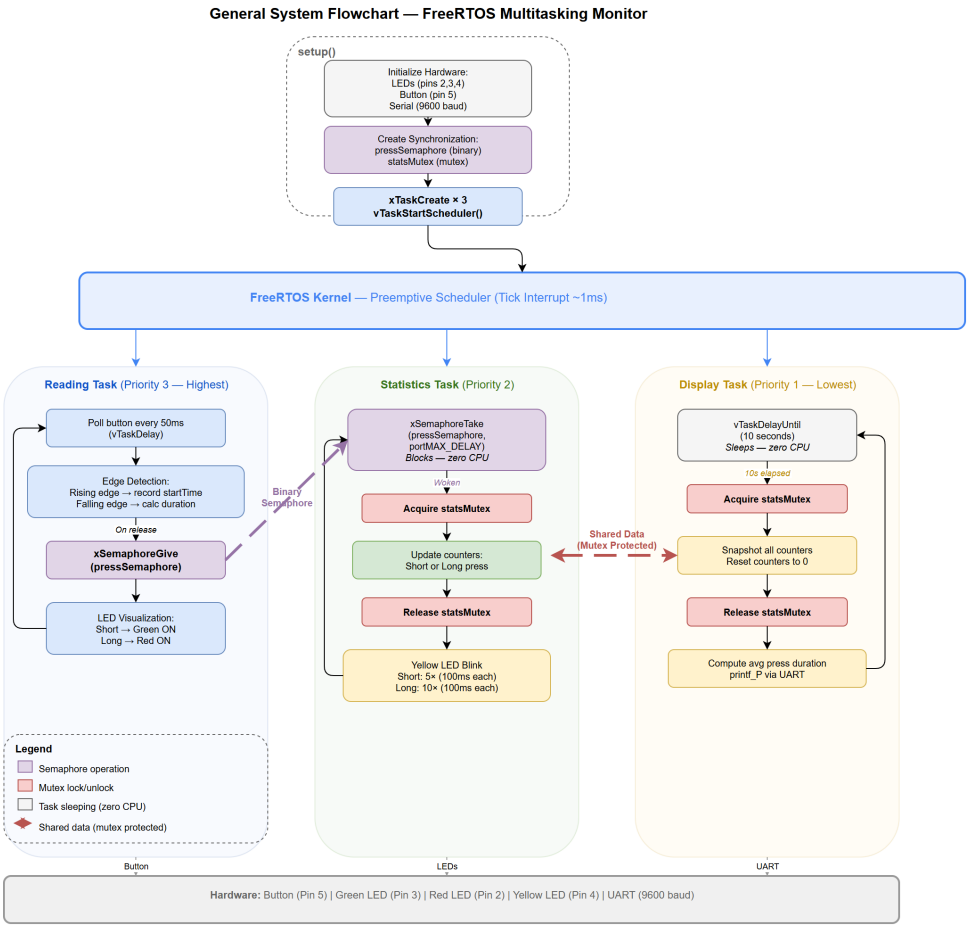


Figure 6: General System Flowchart

3 Implementation and Results

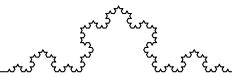
3.1 Software Layering

The code is organized into a clean directory structure. The FreeRTOS library is included as a PlatformIO dependency, providing the kernel, semaphore, and task management APIs. The `src/labs/lab2_2/src/` directory contains the three task implementations, each in its own `.h/.cpp` pair, along with `main.cpp` which initializes hardware, creates tasks, and starts the scheduler. The shared hardware drivers (Led, Button, SerialIO) are reused from the common library.

3.2 Task-Specific Implementation Details

3.2.1 Reading and Measurement

The `ReadingTask.cpp` implements edge detection using a `lastState` variable, with 50ms polling via `vTaskDelay(pdMS_TO_TICKS(50))` which also provides implicit debouncing. On a rising edge (press start), `millis()` is recorded. On a falling edge (release),



the duration is calculated and stored in `ReadingTask::lastDuration`. The binary semaphore `pressSemaphore` is then given to wake the Statistics Task, and the LED visualization is updated (green for < 500ms, red for ≥ 500ms).

3.2.2 Global Statistics Management

The `StatisticsTask.cpp` blocks on `xSemaphoreTake(pressSemaphore, portMAX_DELAY)`, making it event-driven rather than polling-based. When woken, it acquires `statsMutex` to safely update the shared counters: separate short/long press counts and cumulative durations. After releasing the mutex, it performs a yellow LED blink sequence (5 blinks for short, 10 for long) using `vTaskDelay()` for timing.

3.2.3 Periodic Reporting

The `DisplayTask.cpp` uses `vTaskDelayUntil()` for drift-free 10-second intervals. It acquires `statsMutex`, snapshots all statistics, resets the counters to zero, then releases the mutex. The average press duration is computed using integer arithmetic to avoid floating-point overhead, formatting results as seconds with two decimal places (e.g., "1.25s").

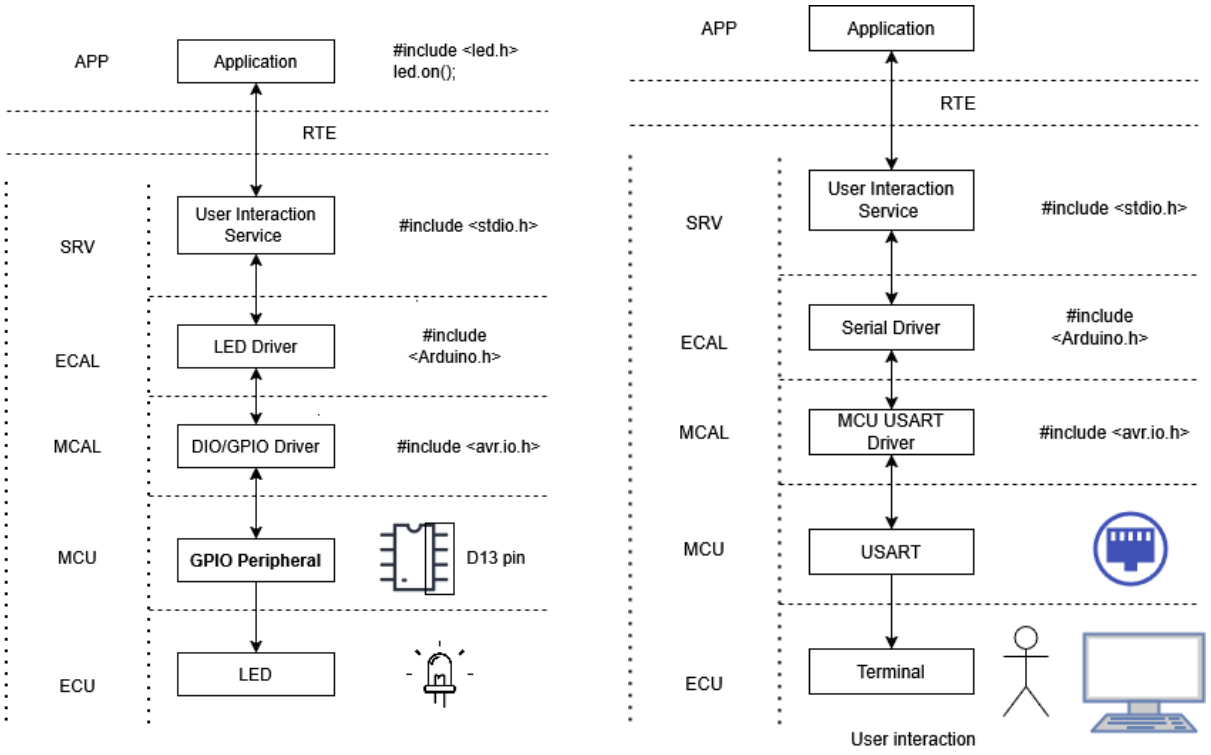
3.3 Hardware Required

The implementation was tested on an ATmega-based development board (e.g., Arduino Uno) with the following hardware components:

- **Button**: Connected to a digital input pin with `INPUT` mode and external resistor.
- **LEDs**: Three LEDs connected to digital output pins with appropriate current-limiting resistors.
- **UART Interface**: The onboard USB-to-serial converter is used for communication with the host computer.

3.4 Layered Design (Hardware-Software)

This project reuses the last laboratory works hardware such as Led, Serial, so they are not repeated in this report, but avaliable in report of lab 1.1 and 1.2 [1, 2]. The diagrams themselves are the following:



(a) LED layering (MCAL/ECAL/APP)

(b) Serial layering (MCAL/ECAL/APP)

Figure 7: Module layering examples for LED and Serial drivers

3.4.1 Button driver

Button driver is new for this lab, and it is implemented in a similar layered manner, represented in figure 8

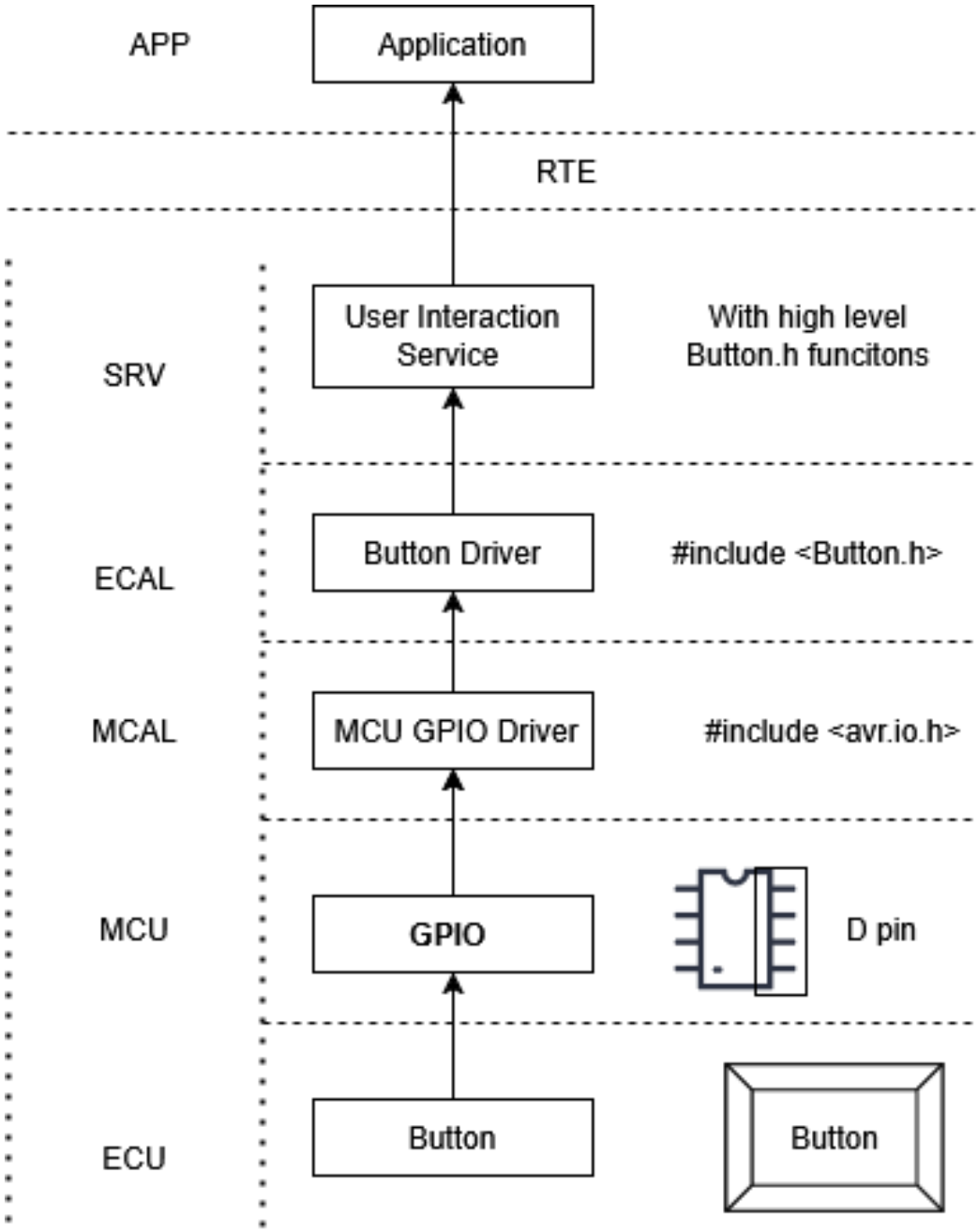


Figure 8: Button layering

The Button driver abstract the hardware details of reading the button state, providing a simple interface for the Reading Task to query the current state. This modular design allows for easy replacement or enhancement of the button handling logic (e.g., adding software debouncing) without affecting the rest of the system.

The layers are the following:

1. **APP** – Application Layer: Calls the service to check button state and reacts accordingly (e.g., trigger an event or update display).

2. **RTE** – Runtime Environment: Routes communication between the application and

the service layer.

3. **SRV** – Service Layer: Contains the User Interaction Service, handles button press detection, debouncing, and state change logic.

4. **ECAL** – ECU Abstraction Layer: Contains the Button Driver (`Button_Driver.h/.c`), provides a `button_read()` function wrapping the GPIO read.

5. **MCAL** – Microcontroller Abstraction Layer: MCU GPIO Driver configures the button pin as `INPUT` using `avr/io.h`, reads pin state via `DDRD` / `PIND` registers.

6. **MCU** – Microcontroller Unit: Physical GPIO pin (e.g., D2 or D7) configured as input, with a pull down resistor for the button connection.

7. **ECU** – Electronic Control Unit: The actual physical push button hardware connected to the MCU pin.

3.5 Electrical Schematics

Electronical circuit of the system is represented in figure 9. It consists of a single button connected to a digital input pin with an external pull down resistor, three LEDs connected to digital output pins with current-limiting resistors, and the UART interface for serial communication. The button is configured to pull the input pin high when pressed, allowing for simple state detection in the Reading Task.

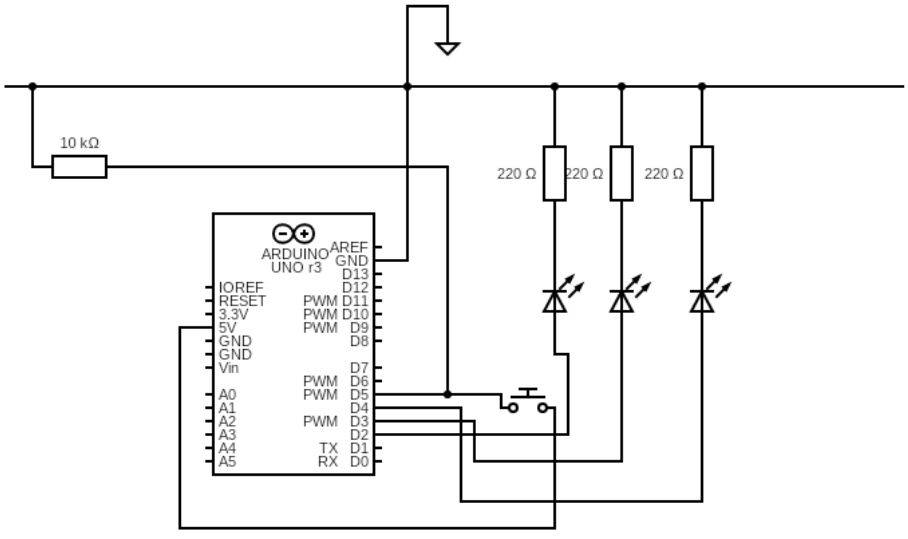


Figure 9: Electrical Schematic of the Multitasking Monitor System

3.6 Simulation

The Simulation of the system will be performed using Wokwi.

A short press followed by a release is shown in figure 10. The Green LED turns on to indicate a short press, and the Yellow LED blinks 5 times rapidly to signal the event. The serial output shows the updated statistics after the next 10-second reporting interval.

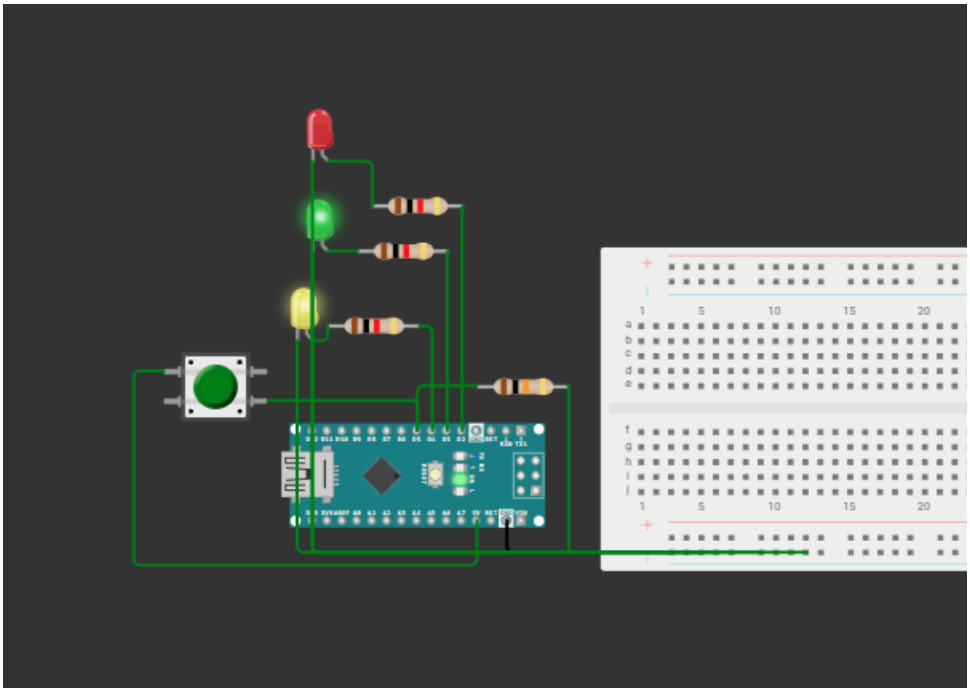


Figure 10: Simulation of a short button press (0.5s)

A long press followed by a release is shown in figure 11. The Red LED turns on to indicate a long press, and the Yellow LED blinks 10 times to signal the event. The serial output shows the updated statistics after the next 10-second reporting interval, reflecting the increased average press duration due to the long press. This demonstrates that the system correctly distinguishes between short and long presses and updates its statistics accordingly.

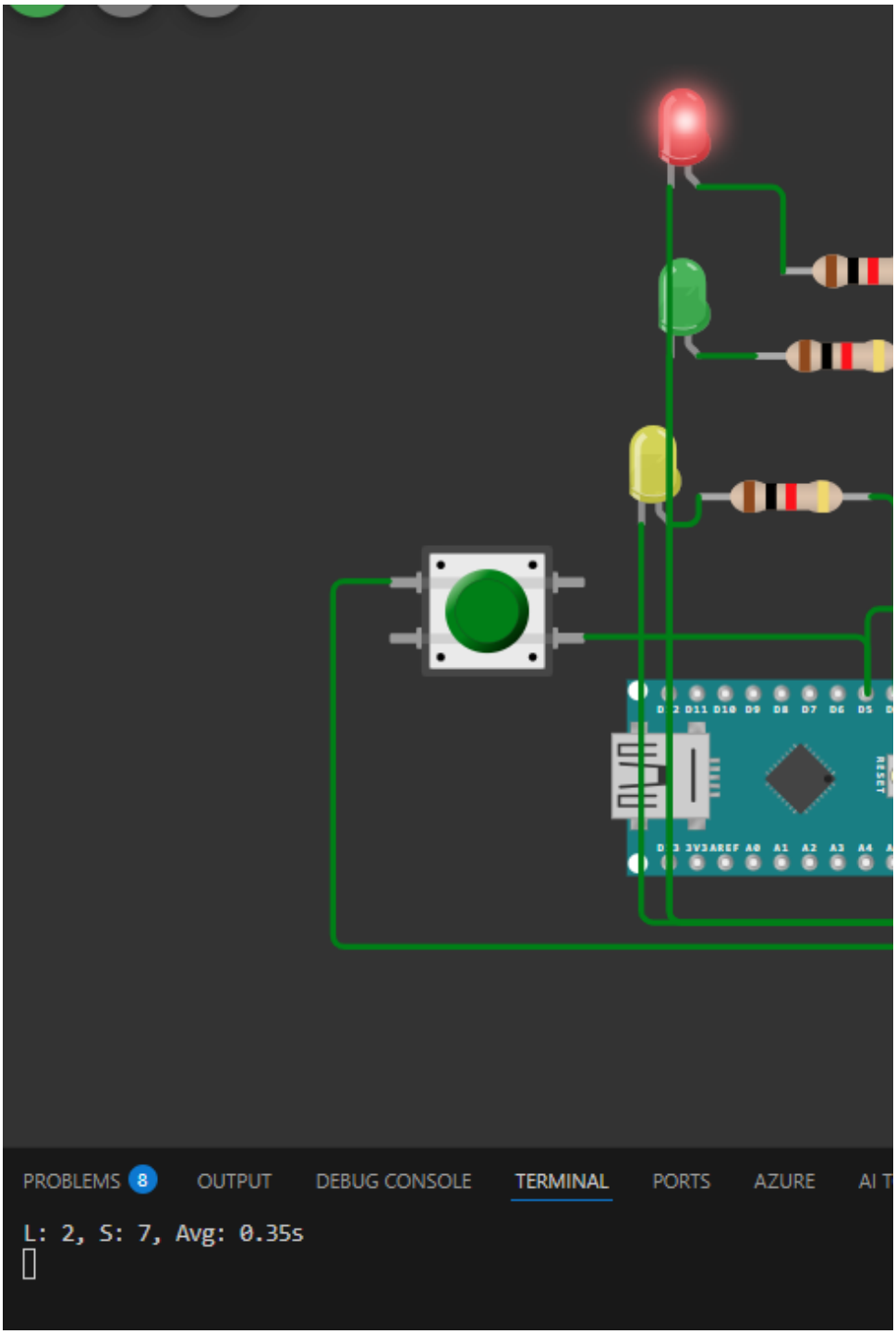


Figure 11: Simulation of a long button press (1.5s)

Next, the statistics are indeed reset after 10 seconds, as shown in figure 12. The serial output shows zeroed counters (e.g., “L: 0, S: 0, Avg: 0.00s”) after the reporting interval in which no new presses occurred, confirming that the Display Task correctly resets the shared statistics under mutex protection.

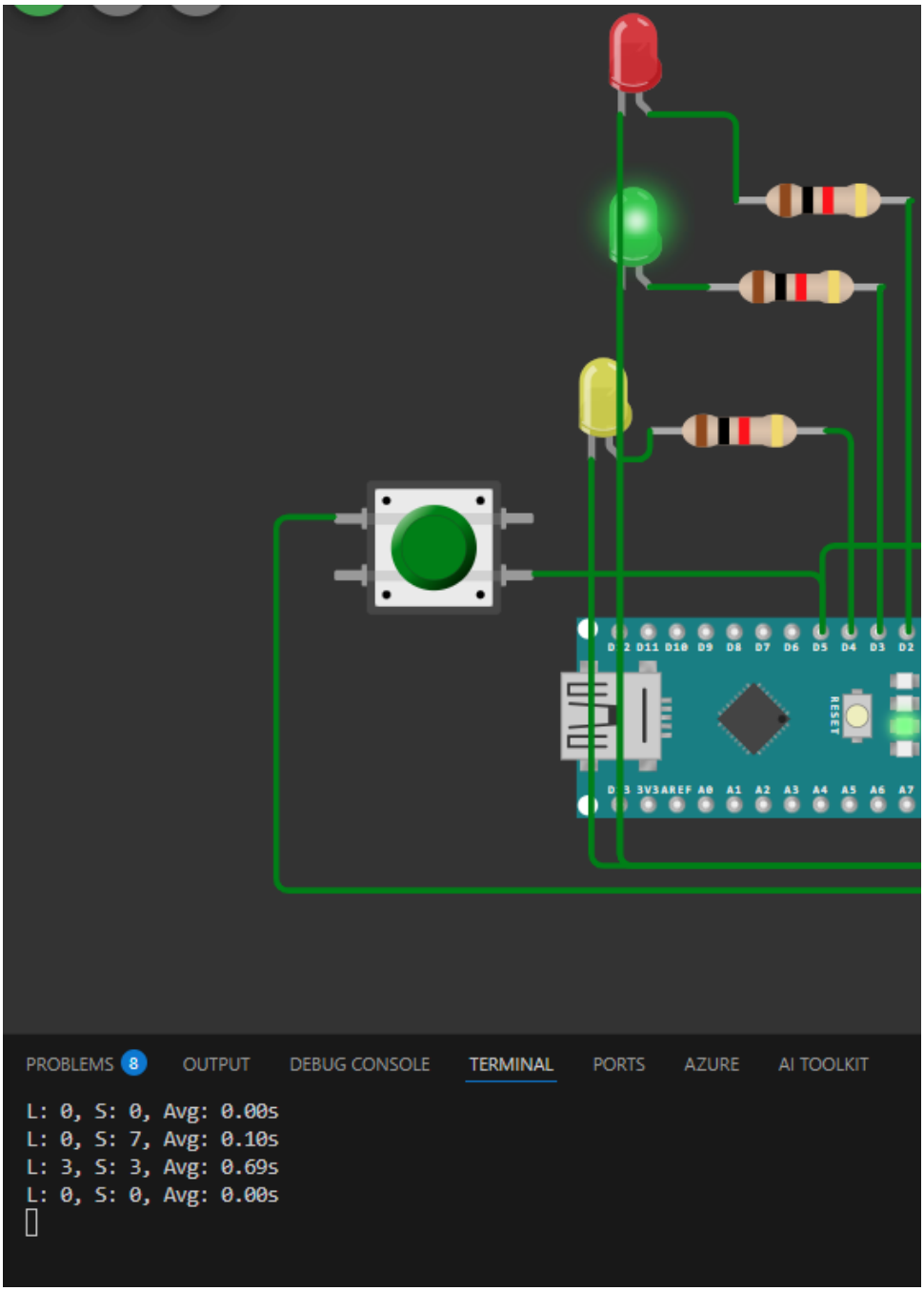


Figure 12: Simulation of statistics reset after 10 seconds

3.7 Physical testing

The real testing happens on Nano board, presented on figure 13. The behavior observed in the physical testing matches the simulation results, confirming that the system correctly handles button presses, updates statistics, and reports via UART as designed.

It works as expected and demonstrates the effectiveness of the FreeRTOS preemptive multitasking approach in an embedded system. The LEDs provide immediate visual feedback, while the serial output allows for detailed monitoring of the system's performance

and statistics over time.

3.8 Challenges

During this laboratory work, several challenges were addressed. The most significant was ensuring correct inter-task synchronization. Initially, shared statistics variables were accessed without mutex protection, leading to occasional data corruption where the Display Task would read partially-updated values. This was resolved by introducing `statsMutex`: the Statistics Task acquires it before updating counters, and the Display Task acquires it before reading and resetting them.

Another challenge was the `extern` declaration placement for global hardware objects (LEDs, button). When declared inside namespace member functions, the C++ compiler resolved them to the task's namespace rather than global scope, causing linker errors. Moving the `extern` declarations to file scope resolved this issue.

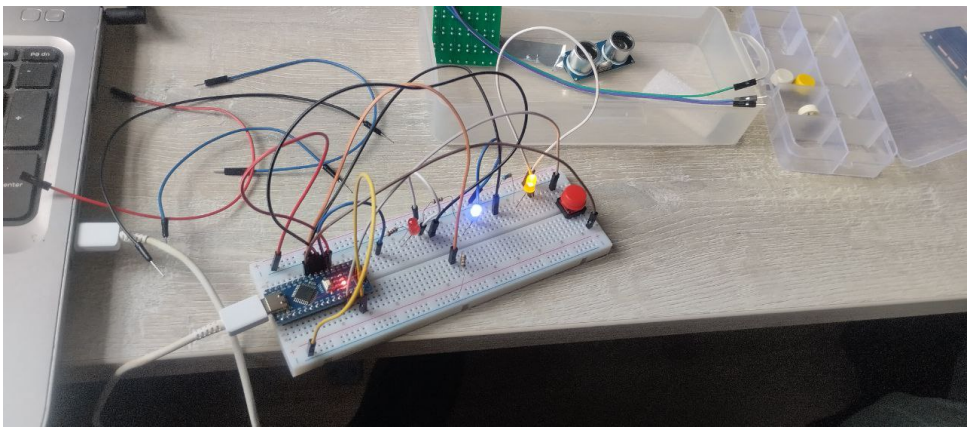


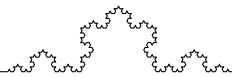
Figure 13: Physical Testing of the Multitasking Monitor System

4 Conclusion

This laboratory work successfully demonstrated the effectiveness of FreeRTOS for managing concurrent tasks on a resource-constrained microcontroller. By leveraging preemptive scheduling, binary semaphores, and mutexes, we implemented a responsive multitasking system that handles button monitoring (50ms polling), event-driven statistics processing, and precise 10-second periodic reporting—all running concurrently with proper synchronization.

Key conclusions include:

- **Preemptive Advantage**: Unlike the cooperative scheduler from Lab 2.1, FreeRTOS allows tasks to block without stalling the system. The Statistics Task sleeps on a semaphore consuming zero CPU, while the Display Task uses `vTaskDelayUntil()` for drift-free periodic execution.



- **Synchronization**: The combination of a binary semaphore for event signaling and a mutex for data protection provides clean, race-free inter-task communication with minimal complexity.
- **Modularity**: The task-based architecture maps naturally to FreeRTOS, with each task (reading, statistics, display) implemented and debugged independently. The shared hardware drivers from previous labs were reused without modification.
- **Trade-offs**: FreeRTOS introduces RAM overhead (per-task stacks, kernel data structures), using approximately 20% of the ATmega328P's 2KB RAM. For more complex systems, this trade-off is justified by significantly simpler application code.

In summary, the implementation met all technical requirements, providing a robust real-time solution for a multitasking monitor system and demonstrating the practical benefits of RTOS-based design over bare-metal approaches in embedded engineering.

References

[1] Lab 1.1 report with Led module implementation and explanation. `https://github.com/TimurCravtov/EmbeddedSystemsLabs/blob/main/reports/l1_1/FAF-231_SI_LAB1-1_Timur-Cravtov.pdf`

[2] Lab 1.2 report with Serial module implementation and explanation. `https://github.com/TimurCravtov/EmbeddedSystemsLabs/blob/main/reports/l1_2/FAF-231_SI_LAB1-2_Timur-Cravtov.pdf`

[3] FreeRTOS, *FreeRTOS Real Time Kernel (RTOS) – API Reference*, `https://www.freertos.org/a00106.html`

[4] Richard Barry, *Mastering the FreeRTOS Real Time Kernel – A Hands-On Tutorial Guide*, `https://www.freertos.org/Documentation/RTOS_book.html`

[5] feilipu, *Arduino FreeRTOS Library*, `https://github.com/feilipu/Arduino_FreeRTOS_Library`

[6] SEI Indrumar Lab 3, Lab 3.2: Sisteme de operare pentru sisteme încorporate – FreeRTOS.

[7] Shared Code Library - Timur Cravtov `https://github.com/TimurCravtov/EmbeddedSystemsLabs`

A Source Code

Besides this appendix, the full source code is available in the GitHub repository [7].

`main.cpp`

```cpp
#include <Arduino_FreeRTOS.h>
#include <semphr.h>
#include <Arduino.h>
#include <stdio.h>

#include <button/button.h>
#include <led/led.h>
#include <serialio/serialio.h>

#include "DisplayTask.h"
#include "ReadingTask.h"
#include "StatisticsTask.h"

// leds
constexpr uint8_t redLedPin = 2;
constexpr uint8_t greenLedPin = 3;
constexpr uint8_t yellowPin = 4;

// button
constexpr uint8_t buttonPin = 5;

// led objects
Led redLed(redLedPin);
Led greenLed(greenLedPin);
Led yellowLed(yellowPin);
Button button(buttonPin);


// magic numbers
extern const uint16_t PRESS_DURATION_THRESHOLD_MS = 500;
extern const uint8_t SHORT_BLINK_NUMBER = 5;
extern const uint8_t LONG_BLINK_NUMBER = 10;

// shared variables
extern SemaphoreHandle_t pressSemaphore;
```

```
36
37 void setup() {
38
39     redLed.init();
40     greenLed.init();
41     yellowLed.init();
42     button.init();
43
44     Serial.begin(9600);
45
46     redirectSerialToStdio(true, true, true);
47
48     pressSemaphore = xSemaphoreCreateBinary();
49
50     xTaskCreate(ReadingTask::run, "Reading", 128, NULL, 3, NULL)
           ;
51     xTaskCreate(StatisticsTask::run, "Statistics", 128, NULL, 2,
           NULL);
52     xTaskCreate(DisplayTask::run, "Display", 128, NULL, 1, NULL)
           ;
53
54     vTaskStartScheduler();
55 }
56
57 void loop() { }
```

Listing 1: main.cpp (lab2_2)

`ReadingTask.h`

```
1  #pragma once
2
3  #include <stdint.h>
4
5  namespace ReadingTask {
6      void run(void* parameters);
7      extern uint16_t recurrenceDelay;
8      extern uint16_t offset;
9      extern uint32_t lastDuration;
10 }
```

Listing 2: ReadingTask.h

ReadingTask.cpp

```cpp
#include <Arduino_FreeRTOS.h>
#include <semphr.h>
#include <Arduino.h>
#include "ReadingTask.h"
#include <button/button.h>
#include <led/led.h>

namespace ReadingTask {
    uint16_t recurrenceDelay = 0;
    uint16_t offset = 0;
    uint32_t lastDuration = 0;
}

void visualizeButtonPressDuration();

SemaphoreHandle_t pressSemaphore;

extern Button button;
extern Led redLed;
extern Led greenLed;

extern const uint16_t PRESS_DURATION_THRESHOLD_MS;

void ReadingTask::run(void* parameters) {

    uint32_t startTime = 0;
    bool lastState = false;

    while (true) {
        bool isPressed = button.isPressed();

        // Button just pressed
        if (isPressed && !lastState) {
            startTime = millis();
        }
```

```
36
37          // Button just released
38          if (!isPressed && lastState && startTime != 0) {
39              ReadingTask::lastDuration = millis() - startTime;
40              startTime = 0;
41
42              xSemaphoreGive(pressSemaphore);
43
44              visualizeButtonPressDuration();
45          }
46
47          lastState = isPressed;
48          vTaskDelay(pdMS_TO_TICKS(50));
49      }
50  }
51
52  void visualizeButtonPressDuration() {
53
54      if (ReadingTask::lastDuration < PRESS_DURATION_THRESHOLD_MS)
            {
55          greenLed.on();
56          redLed.off();
57      } else {
58          redLed.on();
59          greenLed.off();
60      }
61  }
```

Listing 3: ReadingTask.cpp

`StatisticsTask.h`

```
1  #pragma once
2
3  #include <stdint.h>
4  #include <Arduino_FreeRTOS.h>
5  #include <semphr.h>
6
7  namespace Statistics {
8      extern uint16_t shortPressesNumber;
9      extern unsigned long shortPressesTotalDuration;
```

```
10
11      extern uint16_t longPressesNumber;
12      extern unsigned long longPressesTotalDuration;
13
14      extern SemaphoreHandle_t statsMutex;
15  }
16
17  namespace StatisticsTask {
18      void run(void* parameters);
19  }
```

Listing 4: StatisticsTask.h

StatisticsTask.cpp

```
1   #include <Arduino_FreeRTOS.h>
2   #include <semphr.h>
3   #include "StatisticsTask.h"
4   #include "ReadingTask.h"
5   #include <led/led.h>
6
7   namespace Statistics {
8       uint16_t shortPressesNumber = 0;
9       unsigned long shortPressesTotalDuration = 0;
10      uint16_t longPressesNumber = 0;
11      unsigned long longPressesTotalDuration = 0;
12      SemaphoreHandle_t statsMutex = xSemaphoreCreateMutex();
13  }
14
15  extern SemaphoreHandle_t pressSemaphore;
16  extern Led yellowLed;
17  extern const uint16_t PRESS_DURATION_THRESHOLD_MS;
18  extern const uint8_t SHORT_BLINK_NUMBER;
19  extern const uint8_t LONG_BLINK_NUMBER;
20
21  void StatisticsTask::run(void* parameters) {
22
23      while (true) {
24
25          if (xSemaphoreTake(pressSemaphore, portMAX_DELAY) ==
              pdTRUE) {
```

```
26              uint32_t duration = ReadingTask::lastDuration;
27
28              xSemaphoreTake(Statistics::statsMutex, portMAX_DELAY
                    );
29
30              if (duration < PRESS_DURATION_THRESHOLD_MS) {
31                  Statistics::shortPressesNumber++;
32                  Statistics::shortPressesTotalDuration +=
                        duration;
33              } else {
34                  Statistics::longPressesNumber++;
35                  Statistics::longPressesTotalDuration += duration
                        ;
36              }
37
38              xSemaphoreGive(Statistics::statsMutex);
39
40              int blinkCount = (duration >=
                    PRESS_DURATION_THRESHOLD_MS) ? LONG_BLINK_NUMBER
                    : SHORT_BLINK_NUMBER;
41              for (int i = 0; i < blinkCount; i++) {
42                  yellowLed.on();
43                  vTaskDelay(pdMS_TO_TICKS(100));
44                  yellowLed.off();
45                  vTaskDelay(pdMS_TO_TICKS(100));
46              }
47
48          }
49      }
50  }
```

Listing 5: StatisticsTask.cpp

`DisplayTask.h`

```
1  #pragma once
2
3  #include <stdint.h>
4
5  namespace DisplayTask {
6      void run(void* parameters);
```

```
7       extern uint16_t recurrenceDelay;
8       extern uint16_t offset;
9   }
```

Listing 6: DisplayTask.h

DisplayTask.cpp

```
1   #include <Arduino_FreeRTOS.h>
2   #include <semphr.h>
3   #include <stdio.h>
4   #include <stdlib.h>
5   #include <Arduino.h>
6   #include "DisplayTask.h"
7   #include "StatisticsTask.h"
8
9   void DisplayTask::run(void* parameters) {
10      TickType_t lastWakeTime = xTaskGetTickCount();
11      const TickType_t interval = pdMS_TO_TICKS(10 * 1000); // 10
            seconds
12
13      while (true) {
14          vTaskDelayUntil(&lastWakeTime, interval);
15
16          xSemaphoreTake(Statistics::statsMutex, portMAX_DELAY);
17
18          uint16_t currentShort = Statistics::shortPressesNumber;
19          uint16_t currentLong = Statistics::longPressesNumber;
20          uint32_t currentShortDuration = Statistics::
                shortPressesTotalDuration;
21          uint32_t currentLongDuration = Statistics::
                longPressesTotalDuration;
22
23          Statistics::shortPressesNumber = 0;
24          Statistics::longPressesNumber = 0;
25          Statistics::shortPressesTotalDuration = 0;
26          Statistics::longPressesTotalDuration = 0;
27
28          xSemaphoreGive(Statistics::statsMutex);
29
30          uint32_t totalCount = currentShort + currentLong;
```

```
31          uint32_t totalDuration = currentShortDuration +
               currentLongDuration;
32
33          if (totalCount > 0) {
34              uint32_t avgMs = totalDuration / totalCount;
35              uint32_t wholeSeconds = avgMs / 1000;
36              uint32_t fractionalSeconds = avgMs % 1000;
37              uint32_t decimals = fractionalSeconds / 10;
38
39              printf_P(PSTR("L: %u, S: %u, Avg: %lu.%02lus\n\r"),
40                  currentLong,
41                  currentShort,
42                  (unsigned long)wholeSeconds,
43                  (unsigned long)decimals);
44          } else {
45              printf_P(PSTR("L: %u, S: %u, Avg: 0.00s\n\r"),
46                  currentLong,
47                  currentShort);
48          }
49      }
50  }
```

Listing 7: DisplayTask.cpp